

DESIGNING DATA-INTENSIVE APPLICATIONS · VISUAL COMPANION

Chapter 4

Encoding and Evolution

How data formats survive rolling upgrades — backward and forward compatibility

Part I: Foundations · Illustrated · Interview-ready

Martin Kleppmann · Source: `claude-skills / ch04-encoding-and-evolution.md`

Table of Contents

1. [Core Idea](#)
2. [Framework: Thrift / Protocol Buffers](#)
3. [Framework: Avro](#)
4. [Backward & Forward Compatibility](#)
5. [Key Concepts](#)
6. [Dataflow Modes](#)
7. [RPC Flaws & Modern Alternatives](#)
8. [Mental Models](#)
9. [Anti-Patterns](#)
10. [Worked Example: Byte Sizes & Schema Changes](#)
11. [Problems This Solves](#)
12. [Connects To \(Cross-Chapter\)](#)
13. [Key Takeaways](#)
14. [Junior SWE Checkpoints](#)

1. Core Idea

Code changes roll out gradually — rolling upgrades server-side, laggard clients elsewhere — so **old and new code, and old and new data formats, coexist** in one running system. Everything hinges on two compatibility properties:

Backward compatibility

Newer code reads data written by older code.

Easy — you know the old format.

Forward compatibility

Older code reads data written by newer code.

Harder — old code must ignore additions it doesn't understand.

Choose an encoding whose schema evolution rules give you both, and you can deploy any component in any order without downtime.

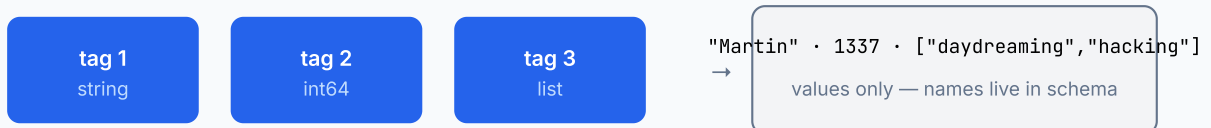
2. Framework: Thrift / Protocol Buffers

Schema-driven binary encodings with numeric **tag numbers**. Internal service-to-service data (RPC, storage) in statically typed shops; compact encoding plus codegen.

When	Internal RPC/storage; statically typed services; you want compact wire format + codegen.
How	Schema (IDL) assigns each field a tag number. Encoded record = concatenation of (tag, type, value) — no field names on the wire .
Why it works	Tags are stable aliases for names — rename freely, wire never sees names. Old code skips unknown tags (type tells it how many bytes to skip) → forward compatible. New code still understands old tags → backward compatible.
Failure mode	Reuse or change a tag number → all existing data silently misinterpreted. Add a <code>required</code> field after v1 → new code fails reading old data. Removed tags are burned forever.

```
required string userName = 1;
optional int64 favorite_number = 2;
repeated string interests = 3;
```

Protobuf on the wire — tags, not names



Old reader sees tag 4 (photoURL) → skip bytes using type annotation

Rename userName → displayName in schema: wire unchanged (tag 1 still tag 1)

Never reuse tag numbers — silent corruption

Figure 2.1 — Tag-based encoding: forward compat by skipping unknown tags; backward compat by reading known tags.

Real systems: gRPC on Protobuf (Google, most microservices), Apache Thrift (Facebook origin), Finagle/Thrift stacks, Rest.li with Protobuf payloads.

3. Framework: Avro

Schema-driven binary encoding with **no tags** — just values concatenated. Most compact of all for the same record.

When	Hadoop-style big files of records; dynamically generated schemas (e.g., dumping a relational DB — column names become field names automatically).
How	Decoder resolves writer's schema (what encoder used) against reader's schema (what decoder expects). They need only be <i>compatible</i> , not identical: fields matched by name, extra writer fields ignored, missing ones filled from reader's default.
Why it works	No tag bookkeeping — friction-free for program-generated schemas. Same Person record: 32 bytes (vs 33 Protobuf, 59 Thrift Binary, 34 Thrift Compact).
Failure mode	Reader must obtain writer's schema somehow — object container file header, schema registry (Espresso), or connection negotiation (Avro RPC). Lose that and bytes are unreadable. Fields without defaults break compatibility on add (backward) or remove (forward).

Avro schema resolution — writer vs reader

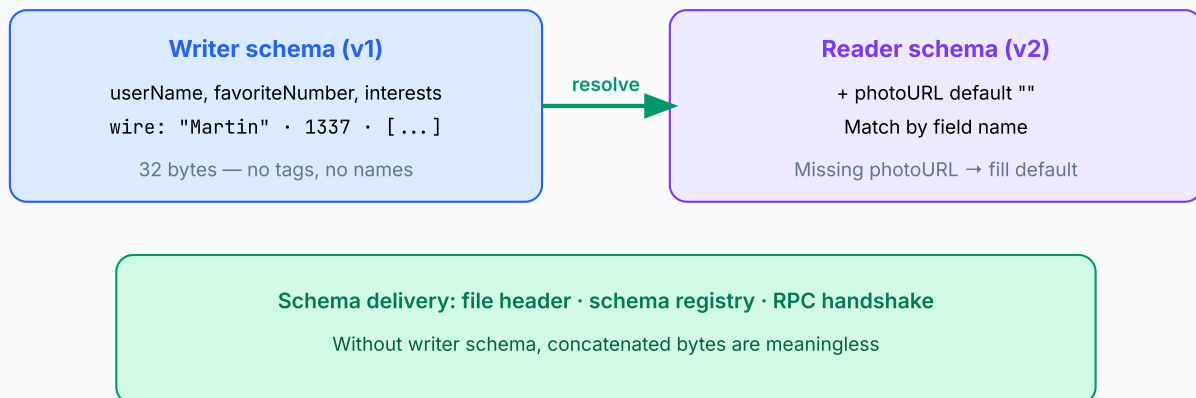


Figure 3.1 — Avro matches fields by name; reader fills defaults for fields absent in writer schema.

Tags vs names is the Protobuf/Avro fork. Tags = wire stability with manual bookkeeping. Name-matching = friction-free dynamically generated schemas. Pick by whether a human or a program authors your schemas.

4. Backward & Forward Compatibility

Change	Protobuf / Thrift	Avro	JSON
Add field	New tag, must be optional/defaulted. Old readers skip tag (forward ✓). New readers tolerate absence (backward ✓).	Add with default value. Old readers ignore; new readers fill default on old data.	Just works if readers ignore extras — but nothing enforces it.
Remove field	Only if optional; never reuse the tag.	Only if field had a default — else forward compat breaks.	Readers may break on missing fields.
Rename field	Free — names aren't on the wire.	Reader-schema aliases: backward but not forward compatible.	Breaks unless both sides agree.
Change datatype	Risky — int32→int64 truncates when old code reads new data. repeated trick for optional scalar→list.	Resolution rules per Avro spec.	Number ambiguity (int vs float, JS 2^53 limit).

Rolling upgrade — both compat directions needed

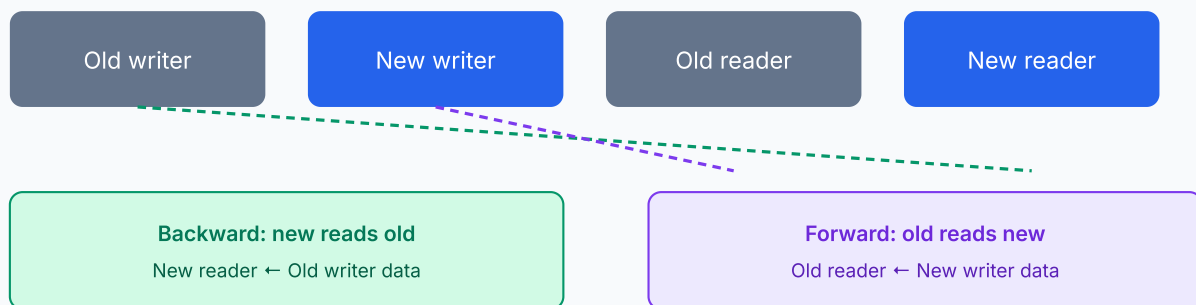


Figure 4.1 — During rolling deploy, all four writer/reader combinations can occur simultaneously.

Decode → model object → re-encode drops unknown fields (Figure 4-7). Old code reads a record, updates it, writes it back — silently deleting fields newer code wrote. Preserve unknown fields at the application level. Same trap when a consumer re-publishes messages.

5. Key Concepts

Encoding formats compared

Format	Pros	Cons
Language-built-in (Java Serializable, pickle)	Convenient	Language-locked, security hole (RCE on decode), versioning-hostile, inefficient. Transient use only.
JSON / XML / CSV	Cross-org interchange, human-readable	Number ambiguity, no binary strings (Base64), optional schemas. Twitter ships 64-bit IDs as strings (JS float corrupts $> 2^{53}$).
Binary JSON (MessagePack)	Modest savings (81→66 bytes)	Field names still ride along — schema-driven formats win by omitting names.
Thrift / Protobuf / Avro	Compact, schema-as-docs, pre-deploy compat checks, codegen	Schema management overhead; Avro needs schema delivery channel.

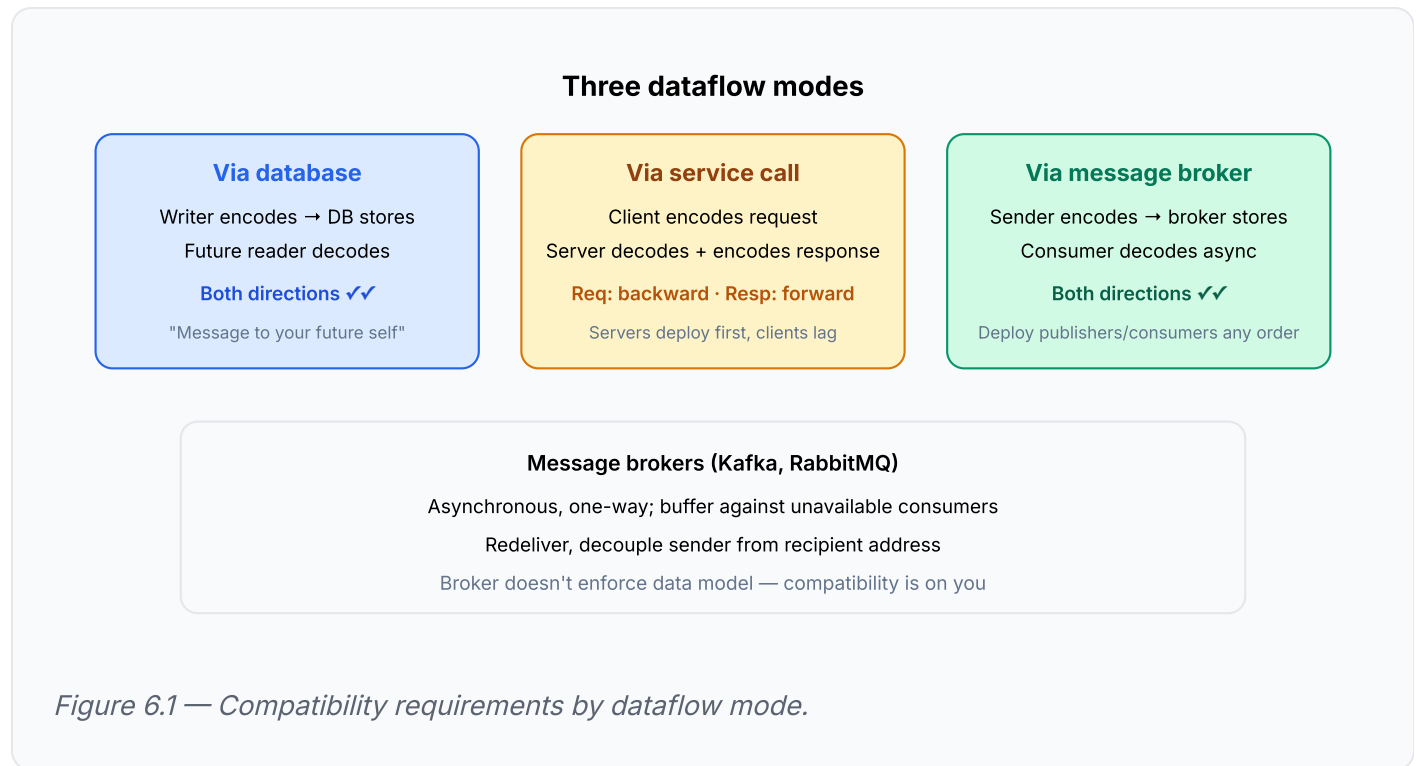
Schema evolution = schema-on-read flexibility with schema-on-write guarantees. Databases fake single-schema appearance — add nullable column, fill nulls on read — while old rows keep their original encoding forever.

Merits of schemas

- **Compactness** — omit field names from every record.
- **Documentation** — schema is guaranteed-current contract.
- **Compatibility checks** — verify forward/backward before deploying.
- **Codegen** — static typing from IDL.

6. Dataflow Modes

Who encodes, who decodes — compatibility responsibilities differ by mode:



Services: servers first, clients second. Backward compatibility on *requests* (old server reads new client fields? No — new server reads old client). Forward compatibility on *responses* (old client reads new server response).

7. RPC Flaws & Modern Alternatives

RPC fakes local calls ("location transparency") but networks add realities code can't ignore:

Network reality	Why RPC hides it at your peril
Timeouts with unknown outcome	Request may have succeeded — retry needs idempotence keys.
Wild latency variance	Local call assumptions break; need futures/async.
No pass-by-reference	Large objects must be copied or referenced by ID.
Retries	Without idempotence, duplicate side effects.

Modern RPC

gRPC on Protobuf, Finagle/Thrift, Rest.li —
embrace the network with futures and streams.
Dominates intra-datacenter.

REST for public APIs

curl-debuggable, no codegen required.
Predominant for cross-org boundaries.

Dead RPC frameworks: CORBA, EJB/RMI, DCOM — no compatibility story. SOAP/WSDL: codegen-dependent, interop-hostile. Distributed actor frameworks on default Java serialization (Akka default) break rolling upgrades — swap in Protobuf.

8. Mental Models

1. Data outlives code

Replace all running code in minutes; five-year-old rows sit in original encoding forever. Migrations are expensive — schema evolution makes the DB *appear* single-schema.

2. Writing to DB = message to future self

Backward compatibility non-negotiable. Forward too — during rolling upgrade, old readers see new writers' data.

3. Tags vs names fork

Protobuf: manual tag bookkeeping, wire-stable renames. Avro: name-matching, program-generated schemas. Pick by schema author.

4. Servers first, clients second

Service deployment assumption → backward-compatible requests, forward-compatible responses.

9. Anti-Patterns

Anti-pattern	Why it fails
Language-native serialization for persistent/cross-service data	Security (RCE), lock-in, no versioning
Adding <code>required</code> field or reusing retired tag after v1	New code can't read old data; silent misinterpretation
Decode → update → re-encode dropping unknown fields	Silently deletes fields newer code wrote (Figure 4-7)
Trusting JSON numbers for 64-bit IDs in JavaScript	Floats corrupt integers $> 2^{53}$ — Twitter uses strings
RPC pretending network isn't there	CORBA/EJB/RMI/DCOM dead; timeouts mean unknown outcome
Akka default Java serialization	Breaks rolling upgrades — use Protobuf
Avro fields without defaults on evolvable schemas	Add breaks backward; remove breaks forward

10. Worked Example: Byte Sizes & Schema Changes

One record: {userName: "Martin", favoriteNumber: 1337, interests: ["daydreaming", "hacking"]}

Encoding	Bytes	What's on the wire
JSON	81	Field names in every record
MessagePack	66	Binary JSON — names still present
Thrift BinaryProtocol	59	Tag + type + value tuples
Thrift CompactProtocol	34	Varints + packed tag/type bytes
Protocol Buffers	33	Tag + type + value (no names)
Avro	32	Pure concatenated values — meaningless without writer schema

Schema change walkthrough

Add photoURL field:

JSON: just works if readers ignore extras — nothing enforces it

Protobuf: new tag 4, must be optional/defaulted

old readers skip tag 4 (forward ✓)

new readers tolerate absence (backward ✓)

Avro: add with default ""

old schema readers ignore it

new schema readers fill default on old data

Remove field:

Protobuf: only if optional; never reuse tag

Avro: only if it had a default (else forward compat breaks)

Rename:

Protobuf: free — names not on wire

Avro: reader-schema aliases — backward but not forward

Byte size comparison — same Person record

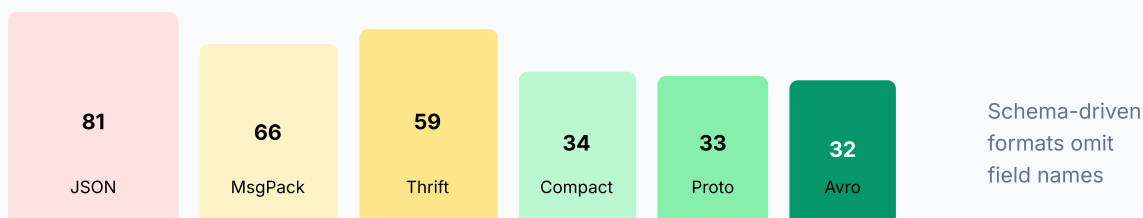


Figure 10.1 — Omitting field names from the wire is the main size win over schemaless formats.

11. Problems This Solves

Lowest-leverage chapter across the repo's 28 system-design examples — only one grounds it directly. Most designs don't surface schema evolution as a named concern even where it's implicitly present (any Kafka-based pipeline).

Mechanism	Problem it solves	Example
Backward/forward-compatible message schema across producer, broker, and consumer	Producers and consumers deploy independently; neither side can assume the other is on the same schema version	Distributed message queue (explicitly requires schema compliance between producer/queue/consumer)

12. Connects To (Cross-Chapter)

Chapter	Connection	Interview soundbite
Ch 1 Evolvability	This chapter is evolvability's data-layer mechanics.	"Schema evolution is how you evolve data formats without downtime."
Ch 2 Data Models	Schema-on-read vs schema-on-write: evolution gives flexibility of the former with guarantees of the latter.	"Avro/Protobuf = enforced schema-on-read flexibility."
Ch 7 Transactions	"Serializability" (transactions) — unrelated to serialization here.	"Don't confuse transaction serializability with data serialization."
Ch 8 Distributed Troubles	Network faults behind RPC's problems — timeouts, unknown outcomes.	"RPC retries need idempotence because of Ch 8 network realities."
Ch 10 Batch Processing	Avro object container files in batch/analytic pipelines.	"Big analytic files use Avro container format with embedded schema."
Ch 11 Stream Processing	Message brokers/Kafka in depth; schema registry patterns.	"Kafka + schema registry = Ch 4 compat at scale."

13. Key Takeaways

1. **Rolling upgrades force coexistence** — design every encoding for backward AND forward compatibility from day one.
2. **Schema-driven binary formats** (Thrift/Protobuf/Avro) beat schemaless JSON internally: smaller, self-documenting, compatibility checkable pre-deploy.
3. **Decision criteria:** cross-org interchange → JSON/REST; internal static-typed services → Protobuf/gRPC or Thrift; dynamically generated schemas and big analytic files → Avro.
4. **Tag numbers are forever:** never reuse, never make post-v1 fields required; in Avro, always give evolvable fields defaults.
5. **Compatibility differs by dataflow mode:** databases need both directions; services need backward-compatible requests, forward-compatible responses; brokers need both to deploy in any order.
6. **Don't let RPC pretend the network isn't there** — retries need idempotence, timeouts mean unknown outcome.

14. Junior SWE Checkpoints

Q1: What's the difference between backward and forward compatibility?

Backward: newer code reads old data (you know the old format). **Forward:** older code reads new data (old code must skip/ignore unknown fields). Both needed during rolling upgrades.

Q2: Why is Protobuf 33 bytes but JSON 81 for the same record?

Protobuf puts tag+type+value on the wire — field names live only in the schema. JSON repeats field names in every record. MessagePack (66 bytes) still carries names.

Q3: You add a required field to a Protobuf schema after v1 is in production. What breaks?

New code reading old data without that field fails. Every post-v1 field must be optional or have a default. Required fields are only safe at initial deployment.

Q4: How does Avro decode bytes with no tags on the wire?

Reader resolves writer's schema against reader's schema (matched by field name). Reader must obtain writer schema via file header, schema registry, or RPC handshake. Without it, bytes are meaningless.

Q5: A service deploys new server before clients update. What compat do you need on requests vs responses?

Requests: backward compatible — new server must read old client requests. **Responses:** forward compatible — old clients must read new server responses (skip unknown fields).

Q6: Why shouldn't old code decode a record to a model object and re-encode it?

Unknown fields get dropped silently — deleting data newer code wrote (Figure 4-7). Same trap when a Kafka consumer re-publishes messages. Preserve unknown fields at application level.

Q7: Why does Twitter ship tweet IDs as JSON strings?

JavaScript numbers are IEEE 754 floats — integers above 2^{53} lose precision. 64-bit IDs must be strings for JS consumers.

Glossary

Term	Definition
Encoding / decoding	Translating in-memory structures ↔ self-contained byte sequences (serialization/marshalling)
Backward compatibility	Newer code reads data written by older code
Forward compatibility	Older code reads data written by newer code
Tag number	Stable numeric field identifier on Protobuf/Thrift wire — never reuse
Writer's schema	Schema used to encode data (Avro)
Reader's schema	Schema expected by decoder — resolved against writer's schema (Avro)
Schema registry	Central store mapping schema version numbers to full schemas
Schema-on-read	Interpret structure at read time (flexible, weak guarantees)
Schema-on-write	Enforce structure at write time (rigid, strong guarantees)
gRPC	Modern RPC on Protobuf with streams and futures
Idempotence	Retry-safe operations — required because RPC timeouts have unknown outcome